长亭科技
CHAITIN

When ASUS IoT Devices Play Hide-and-Seek with Security：A Vulnerability Adventure

@bestswngs @chumen777 @fxc233233

2025 CHAITIN

About us

swing

WeiMing Shi (@swing) is a security researcher from Chaitin Tech. His work primarily focuses on the discovery and exploitation of vulnerabilities, with research interests spanning a wide range of areas, including but not limited to the Internet of Things (IoT) and network security application. He has discovered multiple vulnerabilities in products from major vendors such as Cisco, ASUS, Huawei, Xiaomi and has received acknowledgments.

chumen77

JuYang Gao(@chumen77) is a security researcher, primarily focusing on vulnerability discovery and exploitation, continuously honing and enhancing his skills. Currently, his main focus is on the security of embedded devices. He has discovered multiple RCE vulnerabilities in various smart devices and has received acknowledgments from multiple manufacturers. He has also presented topics at HITB.

fxc233

He is a senior from X1cT34m Team at Nanjing University of Posts and Telecommunications, mainly focuses on the IOT security and protocol security. He has found and reported many vulnerabilities in IoT devices, and has won multiple awards in related competitions as a member of ChaMd5.

CONTENTS

01 Introduction

02 Historical Research

03 Vulnerability Discovery

04 Summary

PART 1

Introduction

Introduction

Why ASUS

ASUS is a big name IoT device maker. They make all kinds of IoT devices — especially routers. But when it comes to security, their routers have had some real problems in the past.

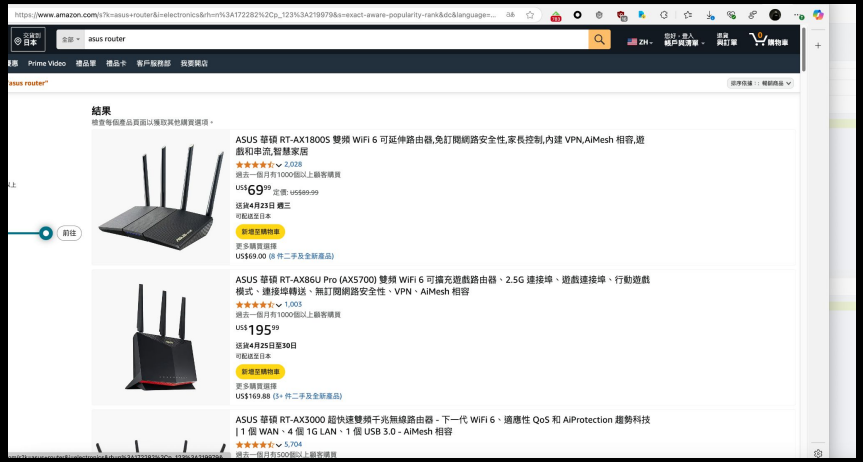
Analysis of ASUS Routers & Vulnerability Chaining

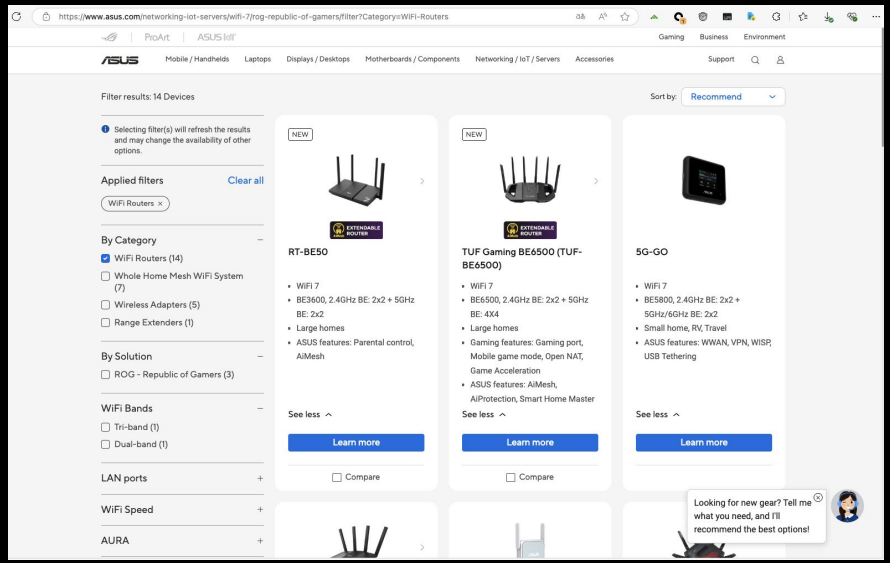
- Overview historical vulnerabilities
  - cfgserver heap overflow
  - httpd auth bypass
- Deep dive into the lighttpd component in AiCloud
  - Identified multiple security risks in this module

Introduction

ASUS IoT WiFI-Router

- ASUS routers have a large market presence
- a wide range of models

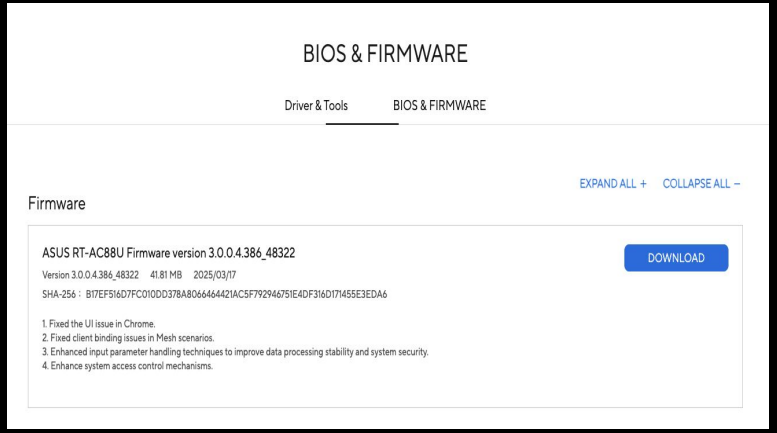




Introduction

Firmware extraction

- The unencrypted firmware can be downloaded from ASUS's official website, which serves as the starting point for our analysis.
- Tools like binwalk and ubif_reader allow for quick extraction and analysis of the firmware contents.







Introduction

Attack Surface Analysis

ASUS routers offer Telnet or SSH services, enabling users to access the underlying system for administrative or debugging purposes.

As a result, we can observe the listening ports of various services on the router.

- httpd port 80 (lan)
- cfg_server port 7788
- …

```
admin@RT-AX56U_V2-A068:/tmp/home/root#
admin@RT-AX56U_V2-A068:/tmp/home/root# netstat -altupn |grep -v "127.0.0.1" |grep "LISTEN"
tcp        0      0 192.168.50.1:49152      0.0.0.0:*               LISTEN      1474/hostapd
tcp        0      0 192.168.50.1:49152      0.0.0.0:*               LISTEN      1471/hostapd
tcp        0      0 0.0.0.0:5152            0.0.0.0:*               LISTEN      347/envrams
tcp        0      0 0.0.0.0:18017           0.0.0.0:*               LISTEN      1402/wanduck
tcp        0      0 192.168.50.1:16363      0.0.0.0:*               LISTEN      2021/uuplugin
tcp        0      0 0.0.0.0:7788            0.0.0.0:*               LISTEN      1656/cfg_server
tcp        0      0 192.168.50.1:80         0.0.0.0:*               LISTEN      1568/httpd
tcp        0      0 192.168.50.1:53         0.0.0.0:*               LISTEN      1503/dnsmasq
tcp        0      0 192.168.50.1:22         0.0.0.0:*               LISTEN      1687/dropbear
tcp        0      0 192.168.50.1:23         0.0.0.0:*               LISTEN      2182/telnetd
tcp        0      0 0.0.0.0:60824           0.0.0.0:*               LISTEN      1857/miniupnpd
tcp        0      0 192.168.50.1:14554      0.0.0.0:*               LISTEN      2021/uuplugin
tcp        0      0 192.168.50.1:48095      0.0.0.0:*               LISTEN      2021/uuplugin
tcp        0      0 :::80                   :::*                    LISTEN      1568/httpd
tcp        0      0 2408:8207:1926:2511::1:53 :::*                    LISTEN      1503/dnsmasq
tcp        0      0 fe80::a236:bcff:fe54:a068:53 :::*                    LISTEN      1503/dnsmasq
tcp        0      0 ::1:53                  :::*                    LISTEN      1503/dnsmasq
tcp        0      0 2408:8207:1926:2511::1:23 :::*                    LISTEN      2180/telnetd
```

PART 2

Historical Security Research

2021 Tianfu Cup cfgserver

cfgserver

Used for networking multiple ASUS routers.

The format of the interaction protocol is TLV, and select the corresponding processing function by comparing tlv_type with the packetHandlers table.

cm_packetProcess

```
if ( v15 <= recv_len )
{
  tlv_type = *recv_data;
  tlv_len = recv_data[1];
  tlv_crc = recv_data[2];
  if ( recv_len - 12 != bswap32(tlv_len) )
  {
    v19 = sub_53BF8((int)"cfg_dbg");
    if ( !strcmp(v19, "1") )
      cprintf("[%s(%d)]:Checking length error !!!\n", "cm_packetProcess", 12472);
    v20 = sub_53BF8((int)"cfg_syslog");
    v21 = strcmp(v20, "1");
    asusdebuglog(6, "cfg_mnt.log", v21, 1, v21, "[%s(%d)]:Checking length error !!!\n", "cm_packetProcess", 12472);
  }
  tlv_data = recv_data + 3;
  v23 = sub_53BF8((int)"cfg_dbg");
  v24 = bswap32(tlv_type);
  if ( !strcmp(v23, "1") )
  {
    v25 = sub_53BB4(v24);
    cprintf("[%s(%d)]:tlv.type(%s)\n", "cm_packetProcess", 12482, v25);
  }
  v26 = sub_53BF8((int)"cfg_syslog");
  if ( !strcmp(v26, "1") )
  {
    v27 = (const char *)sub_53BB4(v24);
    asusdebuglog(6, "cfg_mnt.log", 0, 1, 0, "[%s(%d)]:tlv.type(%s)\n", "cm_packetProcess", 12482, v27);
  }
  v28 = 0;
  while ( 1 )
  {
    v29 = &packetHandlers[v28];
    v30 = packetHandlers[v28];
```

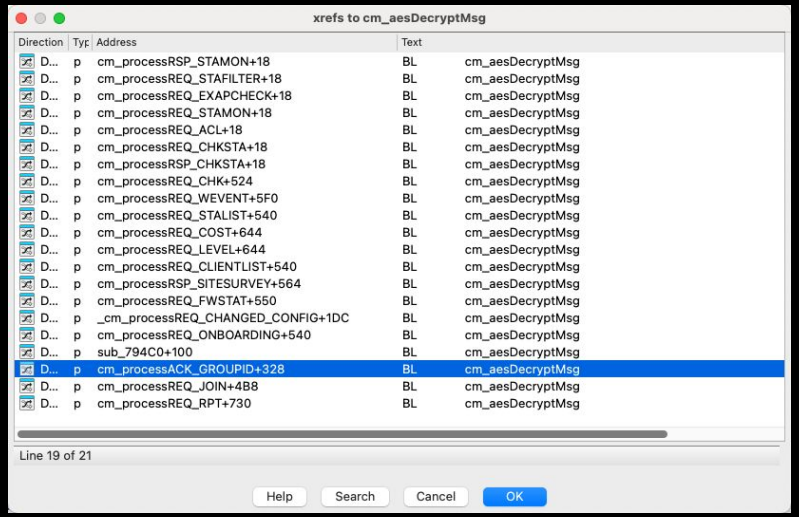
packetHandlers table

```
.data:000AEB8C 14 00 00 00              DCD 0x14
.data:000AEB90 5C CE 05 00              DCD sub_5CE5C
.data:000AEB94 17 00 00 00              DCD 0x17
.data:000AEB98 B4 D4 05 00              DCD sub_5D4B4
.data:000AEB9C 19 00 00 00              DCD 0x19
.data:000AEBA0 0C DB 05 00              DCD sub_5DB0C
.data:000AEBA4 1B 00 00 00              DCD 0x1B
.data:000AEBA8 94 E5 05 00              DCD sub_5E594
.data:000AEBAC 1D 00 00 00              DCD 0x1D
.data:000AEBB0 88 E6 06 00              DCD sub_6E688
.data:000AEBB4 22 00 00 00              DCD 0x22
.data:000AEBB8 48 EF 05 00              DCD sub_5EF48
.data:000AEBBC 24 00 00 00              DCD 0x24
.data:000AEBC0 14 21 06 00              DCD sub_62114
.data:000AEBC4 26 00 00 00              DCD 0x26
.data:000AEBC8 70 7B 07 00              DCD sub_77B70
.data:000AEBCC 28 00 00 00              DCD 0x28
.data:000AEBD0 80 CF 06 00              DCD cm_processREQ_GROUPID
.data:000AEBD4 2A 00 00 00              DCD 0x2A
.data:000AEBD8 7C 98 07 00              DCD cm_processACK_GROUPID
.data:000AEBDC 2B 00 00 00              DCD 0x2B
.data:000AEBE0 3C C4 05 00              DCD sub_5C43C
.data:000AEBE4 2D 00 00 00              DCD 0x2D
.data:000AEBE8 C0 15 06 00              DCD sub_615C0
.data:000AEBEC 2F 00 00 00              DCD 0x2F
```

2021 Tianfu Cup cfgserver

cfgserver

In aes_decrypt(), the parameter of the malloc function is determined by EVP_CIPHER_CTX_block_size(ctx) + length. When length=0xfffffffa is forged, an integer overflow will occur, causing malloc() to apply for a very small piece of memory, resulting in a heap overflow when EVP_DecryptUpdate() is called in a loop to write data to the memory.



The cross-references for the aes_decrypt function

```
cipher = (_DWORD *)EVP_aes_256_ecb();
v10 = (void *)EVP_DecryptInit_ex(ctx, cipher, 0, key, 0);
if ( v10 )
{
  *a4 = 0;
  v11 = EVP_CIPHER_CTX_block_size(ctx) + length;
  v12 = malloc(v11);
  v10 = v12;
  if ( v12 )
  {
    memset(v12, 0, v11);
    out_data_ptr = (int)v10;
    for ( i = length; ; i -= 0x10 )
    {
      in_data_ptr = a2 + length - i;
      if ( i <= 0x10 )
        break;
      if ( !EVP_DecryptUpdate(ctx, out_data_ptr, (int)tmp_out_len, in_data_ptr, 16) )
      {
        printf("%s(%d):Failed to EVP_DecryptUpdate()!!\n", "aes_decrypt", 795);
        EVP_CIPHER_CTX_free(ctx);
        free(v10);
        return 0;
      }
      out_data_ptr += tmp_out_len[0];
      *a4 += tmp_out_len[0];
    }
    if ( i )
    {
      if ( !EVP_DecryptUpdate(ctx, out_data_ptr, (int)tmp_out_len, in_data_ptr, i) )
      {
        printf("%s(%d):Failed to EVP_DecryptUpdate()!!\n", "aes_decrypt", 811);
        EVP_CIPHER_CTX_free(ctx);
        free(v10);
        return 0;
      }
      out_data_ptr += tmp_out_len[0];
      *a4 += tmp_out_len[0];
    }
    if ( !EVP_DecryptFinal_ex(ctx, out_data_ptr, tmp_out_len) )
    {
```

aes_decrypt function code

CVE-2021-32030 httpd auth bypass

http handle request

The code was obtained from ASUS's publicly released GPL source code.

```
static void
handle_request(void)
{
    ...

handler->auth(auth_userid, auth_passwd, auth_realm);
auth_result = auth_check(auth_realm, authorization, url, file, cookies, fromapp); // <---- call to auth_check in
web_hook.o
if (auth_result != 0)
{
    if(strcasecmp(method, "post") == 0 && handler->input)    //response post request
        while (cl--) (void)fgetc(conn_fp);
        send_login_page(fromapp, auth_result, url, file, auth_check_dt, add_try);
        return;
}
```

CVE-2021-32030 httpd auth bypass

http auth_check function

- Check that the request cookies contain asus_token
- Check if the extracted asus_token exists within the current session list
- Check if the extracted asus_token is a stored service token (IFTTT/Alexa)

```
28      user_token[v15] = 0;
29    if ( search_token_in_list(user_token, 0) || check_ifttt_token(user_token) )
30    {
31      if ( !strncmp(auth_realm, "get_IFTTTPincode.cgi", 0x14u) || !strncmp(auth_realm, "get_IFTTTtoken.cgi", 0x12u)
32        add_ifttt_flag();
33      v10 = cur_login_ip_type;
34      if ( !cur_login_ip_type )
35      {           _DWORD
36        ...
37      }
38      nvram_set_int(&HTTPD_LOGIN_FAIL_WAN, 0);
39      nvram_set_int(&HTTPD_LAST_LOGIN_FAIL_TS_W, 0);
40      lock_flag &= ~2u;
41      return 0;
42    }
43    if ( is_firsttime() )
```

CVE-2021-32030 httpd auth bypass

http check_ifttt_token

when the token is Null character (0x0), the User-Agent matches an internal service UA (asusrouter-xxx-ifttt), and no ifttt_token is configured, causing the server to incorrectly identify the request as authenticated.

```
1  int __fastcall check_ifttt_token(const char *asus_token)
2  {
3    // [COLLAPSED LOCAL DECLARATIONS. PRESS NUMPAD "+" TO EXPAND]
4
5    v2 = &GLOBAL_OFFSET_TABLE_;
6    if ( *asus_token
7      && *(_BYTE *)nvram_safe_get("ifttt_token", v1)// if nvram_safe_get return \0
8      && (ifttt_token = (const char *)nvram_safe_get("ifttt_token", v1), !strcmp(asus_token, ifttt_token)) )
9    {
10     if...
11     return 1;                                    // // <----- set result value
12   }
```

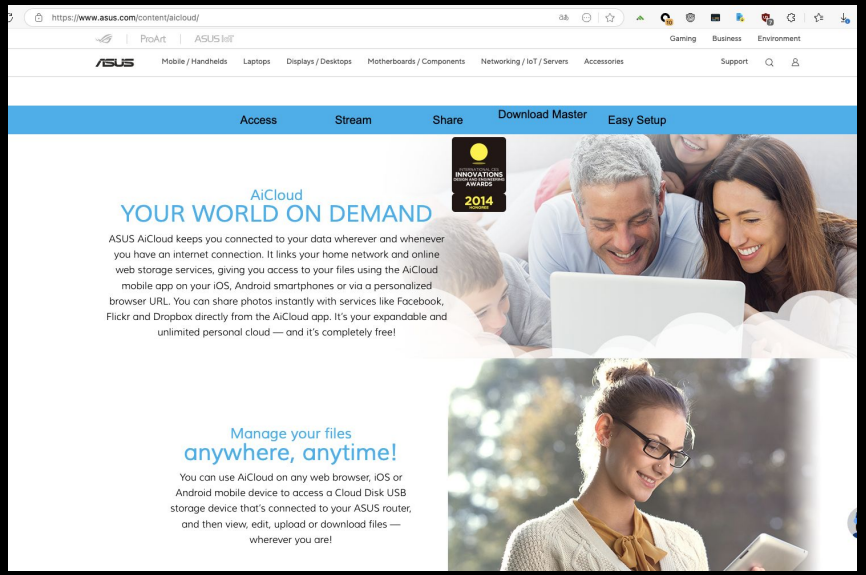
PART 3

Vulnerability Discovery and impact

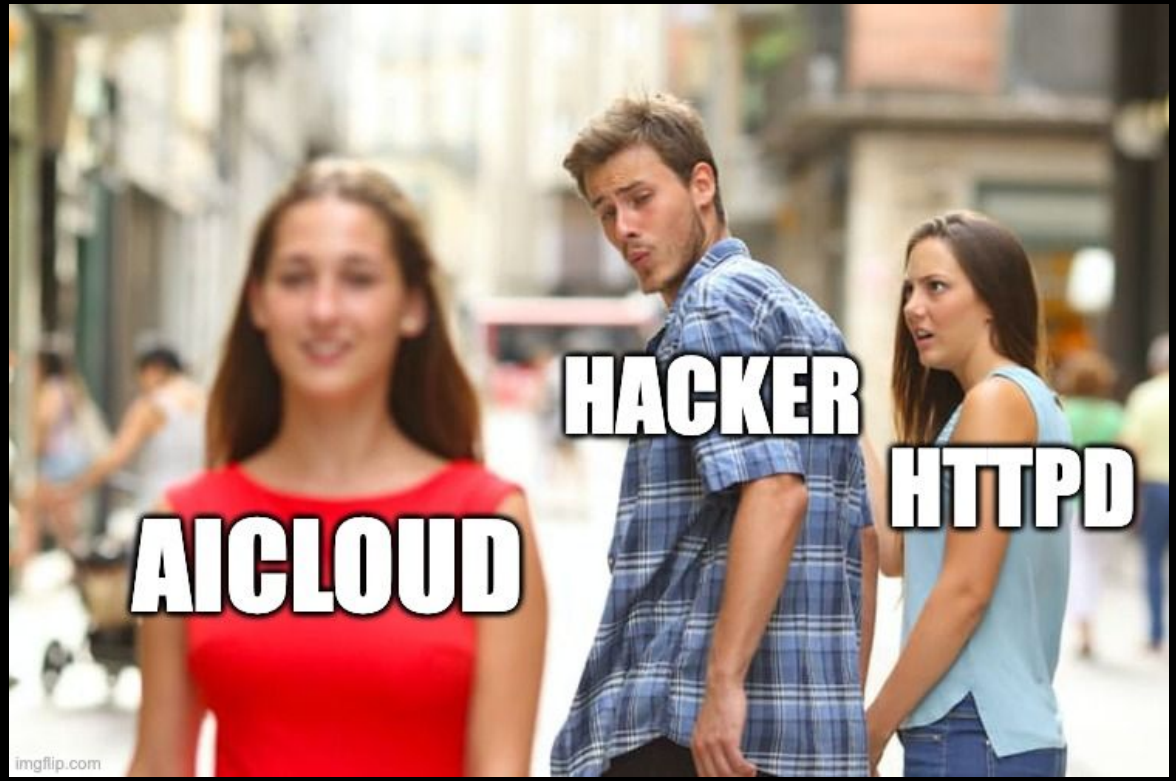
Vulnerability Discovery and impact

Following an analysis of the HTTPD service and other default services bound to listening ports, we shifted our focus to identifying additional attack surfaces.

Following this, the AiCloud functionality emerged as a prominent feature worth further investigation.



Vulnerability Discovery and impact



Vulnerability Discovery and impact

lighttpd aicloud serivice

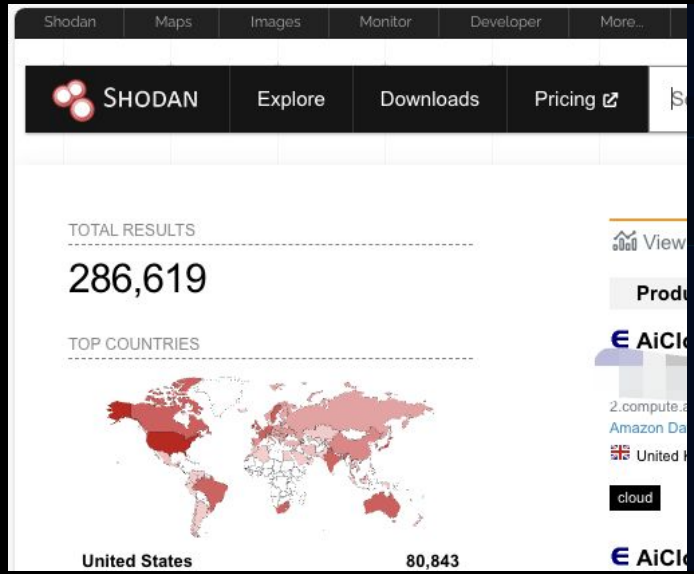
By enabling AiCloud, users can share USB storage devices over the internet. Once a USB device is connected to an ASUS router, files can be accessed remotely through the AiCloud mobile app or any web browser.

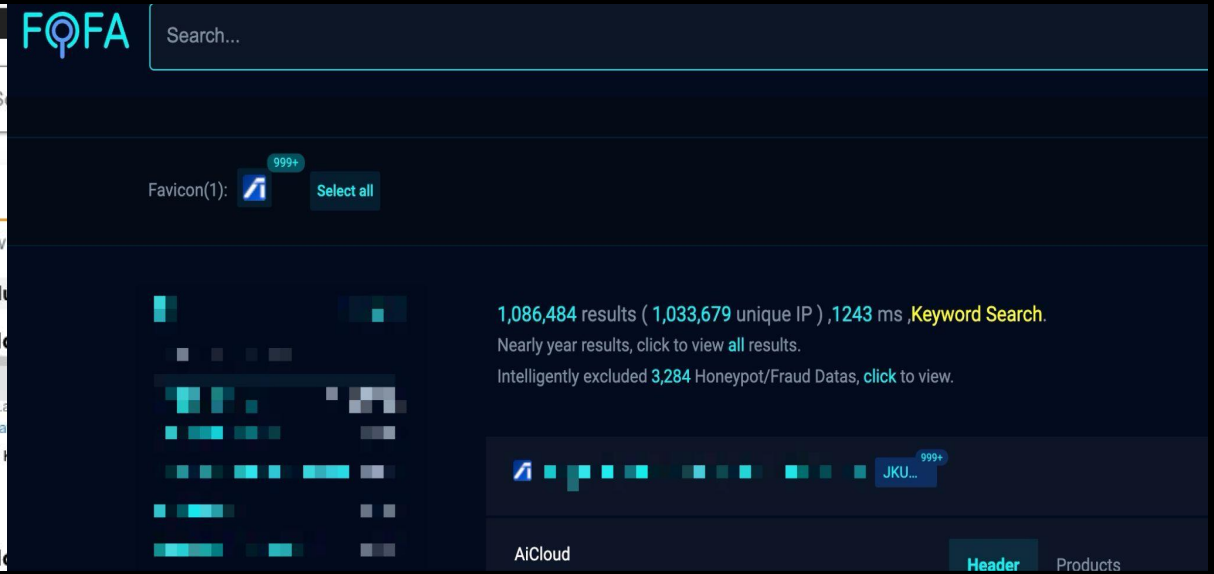


Vulnerability Discovery and impact

lighttpd aicloud serivice

So many ASUS routers AiCloud WebUI exposed





Vulnerability Discovery and impact

Attack Surface Analysis

- httpd port 80
- cfg_server port 7788
- lighttpd port 443

```
udp        0      0 127.0.0.1:43000         0.0.0.0:*                           1521/eapd
admin@RT-AX86U-4D48:/tmp/home/root# netstat -altupn | grep LISTEN | grep -v "127.0.0.1"
tcp        0      0 172.16.64.69:49152      0.0.0.0:*               LISTEN      1548/hostapd
tcp        0      0 172.16.64.69:49152      0.0.0.0:*               LISTEN      1545/hostapd
tcp        0      0 0.0.0.0:5152            0.0.0.0:*               LISTEN      357/envrams
tcp        0      0 0.0.0.0:18017           0.0.0.0:*               LISTEN      1481/wanduck
tcp        0      0 0.0.0.0:7788            0.0.0.0:*               LISTEN      2394/cfg_server
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN      1782/httpd
tcp        0      0 0.0.0.0:8082            0.0.0.0:*               LISTEN      1661/lighttpd
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      1732/dropbear
tcp        0      0 0.0.0.0:443             0.0.0.0:*               LISTEN      1661/lighttpd
tcp        0      0 :::22                   :::*                    LISTEN      1732/dropbear
admin@RT-AX86U-4D48:/tmp/home/root#
```

Vulnerability Discovery and impact

Lighttpd Surface Analysis

lighttpd is a lightweight web server commonly used in embedded devices (such as routers) due to its low memory footprint and modular architecture. When analyzing its attack surface, the following areas are particularly relevant:

1. The HTTP request processing mechanism of lighttpd
2. Loaded Modules (mod_*.so)

Vulnerability Discovery and impact

lighttpd aicloud serivice

Start command, there is a lightttpd configuration item

```
ASUSWRT-Merlin RT-AX86U 3004.388.8_4 Sun Nov 17 19:18:01 UTC 2024
admin@RT-AX86U-4D48:/tmp/home/root# ps w |grep lightt
 1661 admin    17192 S    /usr/sbin/lighttpd -f /tmp/lighttpd.conf -D
 1662 admin    12992 S    /usr/sbin/lighttpd-monitor
 2350 admin    13568 S    /usr/sbin/lighttpd-arpping -f br0
30925 admin     3424 S    grep lightt
admin@RT-AX86U-4D48:/tmp/home/root#
```

```
admin@RT-AX86U-4D48:/tmp/home/root# cat /tmp/lighttpd.conf |grep modu
server.modules+=("mod_aicloud_auth")
server.modules+=("mod_alias")
server.modules+=("mod_aidisk_access")
server.modules+=("mod_aicloud_sharelink")
server.modules+=("mod_query_field_json")
server.modules+=("mod_webdav")
server.modules+=("mod_smbdav")
server.modules+=("mod_compress")
server.modules+=("mod_access")
server.modules+=("mod_auth")
admin@RT-AX86U-4D48:/tmp/home/root#
```

We can see that `lighttpd.conf` loads multiple modules.

Vulnerability Discovery and impact

lighttpd aicloud serivice

The mod of lighttpd is its module system, which is also abbreviated as modules (modules).

These modules provide expandable features for lighttpd, such authentication, and so on.

```
admin@RT-AX86U-4D48:/tmp/home/root# cat /proc/$(pidof lighttpd)/maps |grep mod_ | awk '{print $6}' | sort | uniq
/usr/lib/mod_access.so
/usr/lib/mod_aicloud_auth.so
/usr/lib/mod_aicloud_sharelink.so
/usr/lib/mod_aidisk_access.so
/usr/lib/mod_alias.so
/usr/lib/mod_auth.so
/usr/lib/mod_compress.so
/usr/lib/mod_dirlisting.so
/usr/lib/mod_indexfile.so
/usr/lib/mod_query_field_json.so
/usr/lib/mod_smbdav.so
/usr/lib/mod_staticfile.so
/usr/lib/mod_webdav.so
admin@RT-AX86U-4D48:/tmp/home/root#
```

Vulnerability Discovery and impact

lighttpd aicloud serivice

The mod of lighttpd is its module system, which is also abbreviated as modules (modules).

These modules provide expandable features for lighttpd, such authentication, and so on.

```
admin@RT-AX86U-4D48:/tmp/home/root# cat /proc/$(pidof lighttpd)/maps |grep mod_ | awk '{print $6}' | sort | uniq
/usr/lib/mod_access.so
/usr/lib/mod_aicloud_auth.so
/usr/lib/mod_aicloud_sharelink.so
/usr/lib/mod_aidisk_access.so
/usr/lib/mod_alias.so
/usr/lib/mod_auth.so
/usr/lib/mod_compress.so
/usr/lib/mod_dirlisting.so
/usr/lib/mod_indexfile.so
/usr/lib/mod_query_field_json.so
/usr/lib/mod_smbdav.so
/usr/lib/mod_staticfile.so
/usr/lib/mod_webdav.so
admin@RT-AX86U-4D48:/tmp/home/root#
```

As several of these modules are directly involved in AiCloud's functionality, they have been selected as part of our attack surface for further analysis.

Vulnerability Discovery and impact

aicloud Surface Analysis

lighttpd is a lightweight web server commonly used in embedded devices (such as routers) due to its low memory footprint and modular architecture. When analyzing its attack surface, the following areas are particularly relevant:

1. The HTTP request processing mechanism of lighttpd
2. Loaded Modules (mod_*.so)

lighttpd connection_state_machine stackoverflow

```
strcpy(needle, "asus_token");
memset(&needle[11], 0, 0x15u);
element = array_get_element((_DWORD *)ctx[76], "Cookie");
if ( element )
{
  v47 = (char **)buffer_init();
  buffer_copy_string_len(v47, **(_DWORD **)(element + 32), *(_DWORD *)(*(_DWORD *)(element + 32) + 4));
  buffer_urldecode_path((int)v47);
  memset(v41, 0, sizeof(v41));
  strncpy(v41, *v47, (size_t)v47[1]);
  buffer_free((void **)v47);
```

A stack buffer overflow vulnerability exists in the `connection_state_machine` when parsing the `Cookie`header in the request.

The solution by Kalmarunionen team for the RWCTF Router4 challenge

lighttpd connection_state_machine stackoverflow

Before calling `strncpy`, a length check of 0x400 was introduced to ensure the buffer size does not exceed the specified limit, preventing potential buffer overflows.

```
element = array_get_element(a2[76], "Cookie");
v81 = element;
if ( element )
{
  v82 = buffer_init(element);
  buffer_copy_string_len(v82, **(_DWORD **)(v81 + 32), *(_DWORD *)(*(_DWORD *)(v81 + 32) + 4));
  buffer_urldecode_path(v82);
  memset(s, 0, sizeof(s));
  v83 = *(_DWORD *)(v82 + 4);
  if ( v83 > 0x400 )
    v83 = 1023;
  strncpy(s, *(const char **)v82, v83);
```

After patch: Added length limit.

Vulnerability Discovery and impact

aicloud Surface Analysis

lighttpd is a lightweight web server commonly used in embedded devices (such as routers) due to its low memory footprint and modular architecture. When analyzing its attack surface, the following areas are particularly relevant:

1. The HTTP request processing mechanism of lighttpd
2. Loaded Modules (mod_*.so)

```
31759 admin     3424 S    grep conf
admin@RT-AX86U-4D48:/tmp/home/root# cat /tmp/lighttpd.conf |grep modu
server.modules+=("mod_aicloud_auth")
server.modules+=("mod_alias")
server.modules+=("mod_aidisk_access")
server.modules+=("mod_aicloud_sharelink")
server.modules+=("mod_query_field_json")
server.modules+=("mod_webdav")
server.modules+=("mod_smbdav")
server.modules+=("mod_compress")
server.modules+=("mod_access")
server.modules+=("mod_auth")
admin@RT-AX86U-4D48:/tmp/home/root#
```

mod_aicloud_auth.so uri stackoverflow

Within the `mod_aicloud_auth.so` module, a stack overflow occurs when processing url.

After patch: Added length limit.

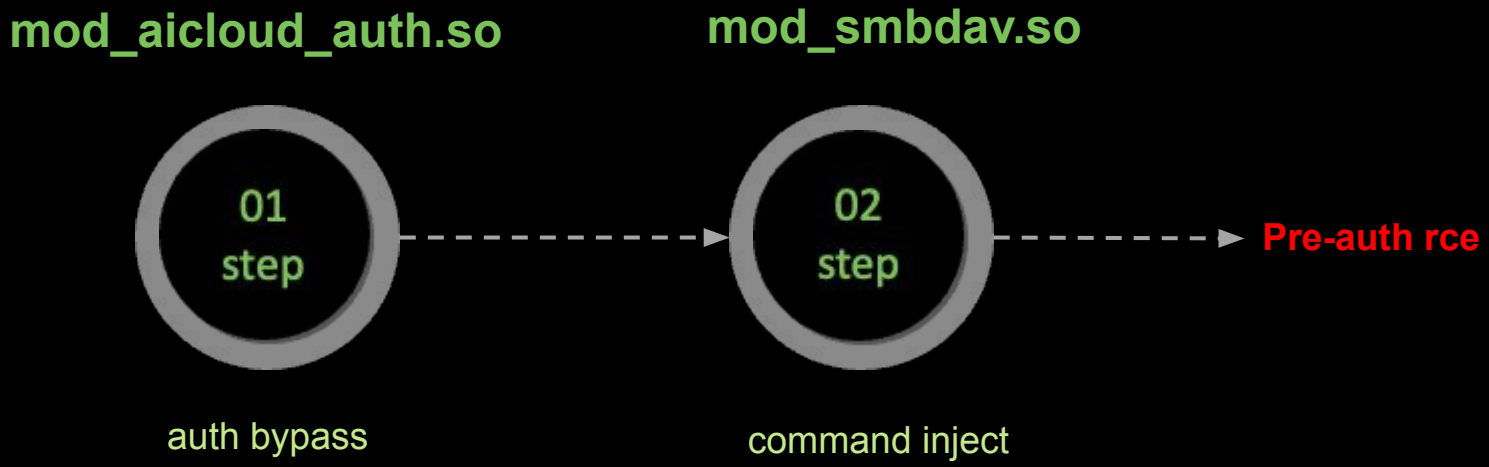
```
size_t v2; // r0
char v5[2048]; // [sp+10h] [bp-101Ch] BYREF
char v6[2048]; // [sp+810h] [bp-81Ch] BYREF
size_t n; // [sp+1010h] [bp-1Ch]
char *s; // [sp+1014h] [bp-18h]

memset(v6, 0, sizeof(v6));
memset(v5, 0, sizeof(v5));
s = strchr(**(const char ***)(a2 + 264), '?');
if ( s )
{
  v2 = strlen(s);
  buffer_copy_string_len(*(_DWORD *)(a2 + 708), s + 1, v2 - 1);
  n = (size_t)&s[-**(_DWORD **)(a2 + 264)];
  strncpy(v5, **(const char ***)(a2 + 264), n);
}
else
{
  strcpy(v5, **(const char ***)(a2 + 264));
}
```

```
memset(v15, 0, sizeof(v15));
memset(v16, 0, 2048);
v2 = **(const char ***)(a1 + 264);
v3 = strchr(v2, '?');
v4 = v3;
if ( v3 )
{
  v5 = strlen(v3);
  buffer_copy_string_len(*(_DWORD *)(a1 + 716), v4 + 1, v5 - 1);
  v6 = **(const char ***)(a1 + 264);
  v7 = v4 - v6;
  if ( (unsigned int)(v4 - v6) >= 0x800 )
    v7 = 2047;
  strncpy(v16, v6, v7);
  v16[v7] = 0;
}
else
{
  v8 = 2047;
  v9 = strlen(v2);
  if ( v9 < 0x7FF )
    v8 = v9;
  strncpy(v16, v2, v8);
  v16[v8] = 0;
}
```

The solution by Bluewater team for the RWCTF Router4 challenge

Vulnerability Chain Analysis: Old school



mod_aicloud_auth.so auth bypass

```c
int plugins_call_handle_physical(server *srv, connection *con) {
    // Iterate through all enabled plugins
    for (plugins = srv->plugins; *plugins; plugins++) {
        // Call each plugin's handler function
        r = p->handle_physical_path(srv, con, p->data);

        // Process the return value to determine next actions
        if (r == HANDLER_FINISHED) {
            // Processing complete, stop calling other plugins
            return HANDLER_FINISHED;
        }
        else if (r != HANDLER_GO_ON) {
            return r;  // Return error code or other special status
        }
        // Continue to the next plugin when HANDLER_GO_ON is returned
    }
    return HANDLER_GO_ON;
}
```

The core plug-in call function of the Lighttpd server is responsible for executing all the processing functions of the enabled plug-ins in turn. Main functions:

1. Traverse all enabled plug-ins in configuration order.
2. Call the handle_physical_path function of each plug-in in turn.
3. Decide whether to continue executing subsequent plug-ins based on the return value
    a. HANDLER_GO_ON(1)：Continue calling the next module
    b. HANDLER_FINISHED(2)：Processing complete, stop calling

mod_aicloud_auth.so auth bypass

```
handler_t mod_aicloud_auth_physical_handler(server *srv, connection *con, plugin_data *p) {
    int auth_result = 0;
    // Check if request has a valid path
    if (!con->uri.path->used) return HANDLER_GO_ON;
    // Get the User-Agent from request headers
    data_string *ds_user_agent = (data_string *)array_get_element(con->request.headers, "user-Agent");
    // [Additional header processing...]
    // Set initial authentication mode to Basic
    con->mode = SMB_BASIC;
    // Determine authentication mode.
    get_aicloud_connection_auth_type(srv, con);
    // Check if authentication mode was set
    if (!con->mode) {
        // [Handle unauthorized case]
        smbc_wrapper_response_401(srv, con);
        return HANDLER_FINISHED;
    }
    // [Special path handling...]
    // Initialize SMB connection information based on request path
    int smb_init_result = aicloud_connection_smb_info_init(srv, con, p);
    if (smb_init_result == HANDLER_UNSET)
        return HANDLER_FINISHED;
    if (smb_init_result == HANDLER_FINISHED) {
        con->http_status = 451;
        return HANDLER_FINISHED;
    }
    // Call the appropriate authentication handler based on chosen mode
    if (con->mode == SMB_NTLM) {
        auth_result = ntlm_authentication_handler(srv, con, p);
    } else if (con->mode == SMB_BASIC) {
        auth_result = basic_authentication_handler(srv, con, p);
    }
    // Process URL path mapping for subsequent handlers
    aicloud_connection_smb_info_url_patch(srv, con);
    // Interpret handler result and determine if processing should continue
    if (auth_result)
        return HANDLER_FINISHED; // Stop processing if result is non-zero
    else
        return HANDLER_GO_ON;    // Continue processing if result is zero
}
```

The core authentication processing function of the AiCloud module, the main process:

1. Determine the authentication mode (NTLM/Basic).
2. Initialize SMB connection information.
3. Call the corresponding authentication processor.
4. Determine whether the request is passed or rejected based on the return value.

mod_aicloud_auth.so auth bypass

```
void get_aicloud_connection_auth_type(server *srv, connection *con) {
    // [Previous code omitted]
    // Check for User-Agent header
    if (NULL == (ds = (data_string *)array_get_element(con->request.headers, "user-Agent")))
        return;
    config_values_t cv[] = {
        { "smbdav.auth_ntlm",     NULL, T_CONFIG_ARRAY, T_CONFIG_SCOPE_CONNECTION },       /*
        { NULL,                   NULL, T_CONFIG_UNSET, T_CONFIG_SCOPE_UNSET }
    };
    // Iterate through all configuration contexts
    for (i = 1; i < srv->config_context->used; i++) {
        // Extract smbdav.auth_ntlm values from all configurations
        // without checking if the context applies to current request
        config_insert_values_global(srv,
            ((data_config *)srv->config_context->data[i])->value,
            cv, ...);
        // Check if User-Agent matches any of the extracted values
        for (j = 0; j < auth_ntlm_array->used; j++) {
            if (strncmp(ds->value->ptr, ds2->value->ptr, len) == 0) {
                con->mode = SMB_NTLM;  // Set NTLM mode
                found = 1;
                break;
            }
        }
    }
}
```

This function determines the authentication type for an AiCloud connection based on the User-Agent header. Key operations:

1. Retrieves the User-Agent from the request headers.
2. Iterates through all configuration contexts in the server.
3. Extracts "smbdav.auth_ntlm" values from configurations.
4. Compares the User-Agent against these values.
5. Sets connection mode to NTLM if a match is found.

The function's role is to decide whether to use NTLM authentication based on the client's User-Agent string.

mod_aicloud_auth.so auth bypass

```
void get_aicloud_connection_auth_type(server *srv, connection *con) {
    // [Previous code omitted]
    // Check for User-Agent header
    if (NULL == (ds = (data_string *)array_get_element(con->request.headers, "user-Agent")))
        return;
    config_values_t cv[] = {
        { "smbdav.auth_ntlm",     NULL, T_CONFIG_ARRAY, T_CONFIG_SCOPE_CONNECTION },       /*
        { NULL,                   NULL, T_CONFIG_UNSET, T_CONFIG_SCOPE_UNSET }
    };
    // Iterate through all configuration contexts
    for (i = 1; i < srv->config_context->used; i++) {
        // Extract smbdav.auth_ntlm values from all configurations
        // without checking if the context applies to current request
        config_insert_values_global(srv,
            ((data_config *)srv->config_context->data[i])->value,
            cv, ...);
        // Check if User-Agent matches any of the extracted values
        for (j = 0; j < auth_ntlm_array->used; j++) {
            if (strncmp(ds->value->ptr, ds2->value->ptr, len) == 0) {
                con->mode = SMB_NTLM;  // Set NTLM mode
                found = 1;
                break;
            }
        }
    }
}
```

```
else $HTTP["url"]=~"^/smb($|/)"{
    server.document-root = "/"
    alias.url=("/smb"=>"/usr/lighttpd")
    smbdav.auth_ntlm = ("Microsoft-WebDAV","xxBitKinex","WebDrive")
    webdav.activate="enable"
    webdav.is-readonly="disable"
}
```

Even if the configuration 'smbdav.auth_ntlm' is only defined in the '/smb' path condition block, it will be applied when accessing other paths such "/"

mod_aicloud_auth.so auth bypass

```
int aicloud_connection_smb_info_init(server *srv, connection *con, plugin_data *p) {
    // Parse path components
    smbc_wrapper_parse_path2(con, pWorkgroup, pServer, pShare, pPath);

    URI_QUERY_TYPE qflag = SMB_FILE_QUERY;  // Default query type
    // [Connection initialization code...]

    // Root path "/" handling - when pServer is empty
    if (pServer[0] == '\0') {
        if (qflag == SMB_FILE_QUERY) {
            // Set to host query mode
            qflag = SMB_HOST_QUERY;
        }
    }
    // [Additional path processing...]
    // Set final query flag
    con->smb_info->qflag = qflag;
    // [Final initialization steps...]
}
```

This function initializes SMB connection information based on the requested URL path. Key operations:

1. Parses URL path components (workgroup, server, share, path).
2. Determines the query type based on available components.
3. For root path "/" (when server name is empty), sets query flag to SMB_HOST_QUERY.
4. Configures connection parameters based on the determined query type.

mod_aicloud_auth.so auth bypass

```
handler_t ntlm_authentication_handler(server *srv, connection *con, plugin_data *p) {
    // [Authentication setup and state checking...]

    if (con->smb_info->qflag == SMB_HOST_QUERY) {
        // Critical issue: No actual authentication in host query mode
        dprintf_impl("smbdav_auth_ntlm.c", "ntlm_authentication_handler",
        238, 1, "host_query do the BASIC authentication");
        goto LABEL_36;
        // Skips the authentication process
    }
    LABEL_36:
    // [Regular authentication flow and logic...]
    dprintf_impl("smbdav_auth_ntlm.c", "ntlm_authentication_handler",
    408, 1, "leave ntlm_authentication_handler");
    // Critical issue: Returns HANDLER_UNSET(0)
    return HANDLER_UNSET;
}
```

This function handles NTLM authentication for SMB connections. Key aspects:

1. At the beginning of the function, check the query flag (qflag) to determine the authentication method.
2. For SMB_HOST_QUERY mode ('/' root path requests), skips actual authentication.
3. For other modes, performs normal NTLM authentication steps.
4. The branch at LABEL_36 returns HANDLER_UNSET(0).

mod_aicloud_auth.so auth bypass

```
handler_t mod_aicloud_auth_physical_handler(server *srv, connection *con, plugin_data *p) {
    // [Previous code omitted]
    // Perform authentication based on selected mode
    if (con->mode == SMB_NTLM) {
        res = ntlm_authentication_handler(srv, con, p);
    } else if (con->mode == SMB_BASIC) {
        res = basic_authentication_handler(srv, con, p);
    }

    // Critical security flaw: Incorrect return value handling
    if (res) {
        return HANDLER_FINISHED;
    }
    // HANDLER_UNSET(0) is incorrectly interpreted as authentication success
    return HANDLER_GO_ON;
}
```

This code snippet shows the critical authentication result handling in the main handler function. Key aspects:

1. Calls the appropriate authentication handler based on the connection mode
2. Examines the return value from the authentication handler
3. Only terminates processing (HANDLER_FINISHED) if the result is NOT HANDLER_UNSET 0
4. **Incorrectly allows** request processing to continue (HANDLER_GO_ON) when result is HANDLER_UNSET

mod_aicloud_auth.so auth bypass

```
handler_t mod_aicloud_auth_physical_handler(server *srv, connection *con, plugin_data *p) {
    int auth_result = 0;

    if (!con->uri.path->used) return HANDLER_GO_ON;
    data_string *ds_user_agent = (data_string *)array_get_element(con->request.headers, "user-Agent");
    // [Request header processing...]
    // Set initial authentication mode to Basic
    con->mode = SMB_BASIC;
    // Determine authentication mode based on User-Agent
    get_aicloud_connection_auth_type(srv, con); //- Point #1
    // Check if authentication mode was set
    if (!con->mode) {
        // [Handle direct connection mode case...]
        smbc_wrapper_response_401(srv, con);
        return HANDLER_FINISHED;
    }
    // [Special path handling...]
    // Initialize SMB connection information - Sets SMB_HOST_QUERY for root path
    int smb_init_result = aicloud_connection_smb_info_init(srv, con, p);
    if (smb_init_result == HANDLER_UNSET)
        return HANDLER_FINISHED;
    if (smb_init_result == HANDLER_FINISHED) {
        con->http_status = 451;
        return HANDLER_FINISHED;
    }
    // Call authentication handler based on mode
    if (con->mode == SMB_NTLM) {
        auth_result = ntlm_authentication_handler(srv, con, p); //- Point #2
    } else if (con->mode == SMB_BASIC) {
        auth_result = basic_authentication_handler(srv, con, p);
    }
    // Process URL path mapping
    aicloud_connection_smb_info_url_patch(srv, con);
    // Interpret authentication result - Point #3
    if (auth_result)
        return HANDLER_FINISHED; // Stop processing only for non-zero values
    else
        return HANDLER_GO_ON;    // HANDLER_UNSET(0) incorrectly interpreted as success
}
```

```
GET / HTTP/1.1
Host: x.x.x.x
User-Agent: Microsoft-WebDAV
...
```

1. get_aicloud_connection_auth_type → sets `con->mode = SMB_NTLM`
point 1: Applies configuration from unrelated context

2. aicloud_connection_smb_info_init → `qflag = SMB_HOST_QUERY`

3. ntlm_authentication_handler →`HANDLER_UNSET(0)`
point 2: Skips authentication for SMB_HOST_QUERY mode

4. mod_aicloud_auth_physical_handler → Incorrectly returns `HANDLER_GO_ON`
point 3: Misinterprets HANDLER_UNSET as authentication success

mod_smbdav.so Vulnerability Discovery

Defined multiple custom HTTP request methods, such as SETROOTCERTIFICATE and APPLYAPP.

These methods are handled by the mod_smbdav_subrequest_handler function in the smbdav module.

```
static keyvalue http_methods[] = {
    { HTTP_METHOD_GET, "GET" },
    { HTTP_METHOD_HEAD, "HEAD" },
    { HTTP_METHOD_POST, "POST" },
    { HTTP_METHOD_PUT, "PUT" },
        ..

    /*Sungmin add*/
        ...
    { HTTP_METHOD_SETROOTCERTIFICATE, "SETROOTCERTIFICATE" },
        ...
    { HTTP_METHOD_APPLYAPP, "APPLYAPP" },
        ...
    { HTTP_METHOD_UNSET, NULL }
};
```

mod_smbdav.so Vulnerability Discovery

Defined multiple custom HTTP request methods, such as SETROOTCERTIFICATE and APPLYAPP.

```
else $HTTP["url"] !~ "^/smb($|/|.)" {
    server.document-root = "smb://"
    smbdav.activate = "enable"
    smbdav.is-readonly = "disable"
    smbdav.always-auth = "enable"
    smbdav.sqlite-db-name =
"/tmp/lighttpd/smbdav.db"
    usertrack.cookie-name = "SMBSESSID"
}
```

The access URL / path will match the configuration rules in this section.

```
static keyvalue http_methods[] = {
    { HTTP_METHOD_GET, "GET" },
    { HTTP_METHOD_HEAD, "HEAD" },
    { HTTP_METHOD_POST, "POST" },
    { HTTP_METHOD_PUT, "PUT" },
        ..
    /*Sungmin add*/
        ...
    { HTTP_METHOD_SETROOTCERTIFICATE, "SETROOTCERTIFICATE" },
        ...
    { HTTP_METHOD_APPLYAPP, "APPLYAPP" },
        ...
    { HTTP_METHOD_UNSET, NULL }
};
```

mod_smbdav.so : SETROOTCERTIFICATE command inject

In the HTTP_METHOD_SETROOTCERTIFICATE handler, the XML payload is parsed to extract values such as key, cert, and intermediate_crt.

```
4331    case HTTP_METHOD_SETROOTCERTIFICATE:
4332      if ( !*(_WORD *)(*(_DWORD *)(a2 + 596) + 120) )
4333      {
4334        ...
4335      }
4336      v503 = 0;
4337      v502 = 0;
4338      v501 = 0;
4339      v340 = 0;
4340      if ( *(_DWORD *)(a2 + 308) && sub_8888(a1, a2, v482, *(_DWORD *)(a2 + 84), &v196) == 1 )
4341      {
4342        v339 = xmlDocGetRootElement(v196);
4343        if ( !v339 )
4344          _assert("rootnode", "mod_smbdav.c", 6385);
4345        if ( !xmlStrcmp(*(_DWORD *)(v339 + 8), "content") )
4346        {
4347          ptr = calloc(1u, 0xCu);
4348          for ( i8 = *(_DWORD *)(v339 + 12); i8; i8 = *(_DWORD *)(i8 + 24) )
4349          {
4350            if ( xmlStrcmp(*(_DWORD *)(i8 + 8), "key") )
4351            {
4352              if ( xmlStrcmp(*(_DWORD *)(i8 + 8), "cert") )
4353              {
4354                if ( !xmlStrcmp(*(_DWORD *)(i8 + 8), "intermediate_crt") )
4355                  v501 = (char *)xmlNodeGetContent(i8);
4356              }
4357              else
4358              {
4359                v502 = (char *)xmlNodeGetContent(i8);
4360              }
4361            }
4362            else
4363            {
4364              v503 = (char *)xmlNodeGetContent(i8);
4365            }
4366          }
4367        }
4368      }
```

mod_smbdav.so : SETROOTCERTIFICATE command inject

However, the cert and intermediate_crt values are passed directly to the system call without sufficient validation, leading to a potential command injection vulnerability.

```
4416    v109 = SSL_CTX_set_verify(v338, 0, 0);
4417    v337 = (const char **)buffer_init(v109);
4418    buffer_copy_string(v337, "echo \"");
4419    buffer_append_string(v337, cert);
4420    buffer_append_string(v337, "\" > /etc/cert.pem.1");
4421    v340 = system(*v337);
4422    if ( v340 < 0 )
4423    {
4424      buffer_free(v337);
4425      v110 = *v337;
4426      v111 = _errno_location();
4427      v112 = strerror(*v111);
4428      dprintf_impl("mod_smbdav.c", "mod_smbdav_subrequest_handler", 6470, 1, "cmd: %s\t error: %s", v110, v112);
4429      *(_DWORD *)(a2 + 128) = 400;
4430      *(_DWORD *)(a2 + 72) = 1;
4431      return 2;
4432    }
4433    if ( SSL_CTX_use_certificate_file(v338, "/etc/cert.pem.1", 1) <= 0 )
4434      goto LABEL_838;
4435    buffer_copy_string(v337, "echo \"");
4436    buffer_append_string(v337, key);
4437    buffer_append_string(v337, "\" > /etc/key.pem.1");
4438    v340 = system(*v337);                       // // <- command inject
```

mod_smbdav.so : APPLYAPP command inject

In the HTTP_METHOD_APPLYAPP handler, specific HTTP headers such as SET_NVRAM and RC_SERVICE are parsed. These values are checked by the is_valid_string function

```
case HTTP_METHOD_APPLYAPP:
        ...
   v471 = array_get_element(*(_DWORD *)(a2 + 304), "SET_NVRAM");
   if ( v471 )
   {
     ...
     if ( is_valid_string(*in_SET_NVRAM) )
     ...
   }
   v471 = array_get_element(*(_DWORD *)(a2 + 304), "RC_SERVICE");
   if ( v471 )
   {
     in_RC_SERVICE = *(char ***)(v471 + 32);
        ...
       if ( is_valid_string(*in_SET_NVRAM) ) // The code was intended to validate
RC_SERVICE, but mistakenly checked in_SET_NVRAM instead
        ...
   }
```

mod_smbdav.so : APPLYAPP command inject

In the HTTP_METHOD_APPLYAPP handler, specific HTTP headers such as SET_NVRAM and RC_SERVICE are parsed. These values are checked by the is_valid_string function

The `is_valid_string` function checks whether a variable is valid by detecting the presence of illegal characters such as the dollar sign ($), semicolon (;), and others.

```
int __fastcall is_valid_string (const char
*a1)
{
  if ( strchr(a1, '"') )
    return -1;
  if ( strchr(a1, '$') )
    return -1;
  if ( strchr(a1, '`') )
    return -1;
  if ( strchr(a1, ';') )
    return -1;
  if ( strchr(a1, '\\') )
    return -1;
  return 0;
}
```

mod_smbdav.so : APPLYAPP command inject

In the HTTP_METHOD_APPLYAPP handler, specific HTTP headers such as SET_NVRAM and RC_SERVICE are parsed. These values are checked by the is_valid_string function

replace_special_chars: Replaces special characters (|, ;, &, $, >, <, ', !) with spaces to sanitize input.

is_valid_string: Validates whether a string contains potentially dangerous characters such as ", $, `, ;, or '

```
_BYTE *__fastcall replace_special_chars(const char *a1, int a2)
{
    ...

  v7 = strlen(a1) + 1;
  qmemcpy(v6, "|;&$><'!\\", 9);
  result = *(_BYTE **)"         ";
  qmemcpy(v5, "          ", 9);
  for ( i = 0; i < v7; ++i )
  {
    for ( j = 0; j <= 8; ++j )
    {
      result = v10;
      if ( a1[i] == v6[j] )
      {
        result = v10;
        *(_BYTE *)(a2 + i) = v5[j];
        break;
      }
    }
    if ( j == 9 )
      *(_BYTE *)(a2 + i) = a1[i];
  }
```

mod_smbdav.so : APPLYAPP command inject

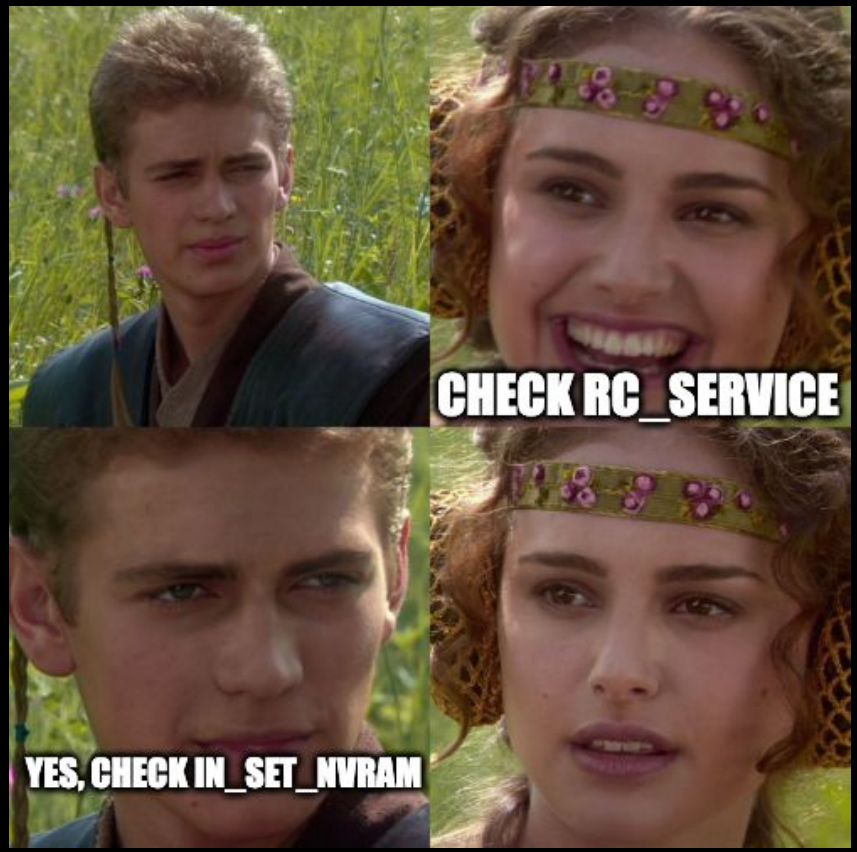
The RC_SERVICE value is sanitized using replace_special_chars, but is then passed directly to system, leading to a potential command injection vulnerability.

```
if ( v488 )
        {
          v327 = (const char **)buffer_init(_in_RC_SERVICE);
          buffer_copy_string(v327, "rc rc_service ");
          buffer_append_string(v327, v488);
          dprintf_impl("mod_smbdav.c", "mod_smbdav_subrequest_handler", 6874, 1,
"rc_command: %s", *v327);
          memset(key, 0, 256);
          replace_special_chars(*v327, (int)key);
          v326 = system((const char *)key); // do system
          if ( v326 >= 0 )
          {
            ...
          }
            ...
        }
```

mod_smbdav.so : APPLYAPP command inject

A variable misuse occurred where `in_SET_NVRAM` was checked instead of the intended `RC_SERVICE`, potentially leading to logic flaws in input validation.

```
  v471 = array_get_element(*(_DWORD *)(a2 + 304),
"RC_SERVICE");
  if ( v471 )
  {
    in_RC_SERVICE = *(char ***)(v471 + 32);
        ...
      if ( is_valid_string(*in_SET_NVRAM) ) // The
code was intended to validate RC_SERVICE, but
mistakenly checked in_SET_NVRAM instead
        ...
  }
```



mod_smbdav.so : APPLYAPP command inject

A variable misuse occurred where `in_SET_NVRAM` was checked instead of the intended `RC_SERVICE`, potentially leading to logic flaws in input validation.

```
  v471 = array_get_element(*(_DWORD *)(a2 + 304),
"RC_SERVICE");
  if ( v471 )
  {
    in_RC_SERVICE = *(char ***)(v471 + 32);
        ...
      if ( is_valid_string(*in_SET_NVRAM) ) // The
code was intended to validate RC_SERVICE, but
mistakenly checked in_SET_NVRAM instead
        ...
  }
```



Chain: Patch the part of the chain.

mod_aicloud_auth.so auth bypass vul patch

```
handler_t ntlm_authentication_handler(server *srv, connection *con, plugin_data *p) {
    // [Previous code omitted]
    // [Authentication setup and state checking...]
    if (con->smb_info->qflag == SMB_HOST_QUERY) {
    {
    dprintf_impl(
      "smbdav_auth_ntlm.c",
      "ntlm_authentication_handler",
      239,
      1,
      "host_query with authorization NTLM is not allowed!!");
    con->http_status = 401;
    return HANDLER_FINISHED;
    }
    // [Subsequent  omitted]
}
```

When matching to a host_query pattern, returns a 401 unauthorized error instead of allowing the request to continue processing.

Chain: Patch the part of the chain.

mod_smbdav.so : SETROOTCERTIFICATE Vuln patch

```
if (is_valid_string(key)) {
    return 400;
}
if (is_valid_string(cert)) {
    return 400;
}
if (is_valid_string(intermediate_cri
    return 400;
}
```

```
int __fastcall is_valid_string (const char
*a1)
{
  if ( strchr(a1, '"') )
    return -1;
  if ( strchr(a1, '$') )
    return -1;
  if ( strchr(a1, '`') )
    return -1;
  if ( strchr(a1, ';') )
    return -1;
  if ( strchr(a1, '\\'') )
    return -1;
  return 0;
}
```

In the updated version of the code, user-controlled fields such as key, cert, and intermediate_crt are all validated using the is_valid_string function.

```
buffer_copy_string(v333, "echo \"");
buffer_append_string(v333, v497);
buffer_append_string(v333, "\" >
/etc/cert.pem.1");
system(*v333);
```

Since the command string is wrapped in double quotes, this patch effectively prevents command injection.

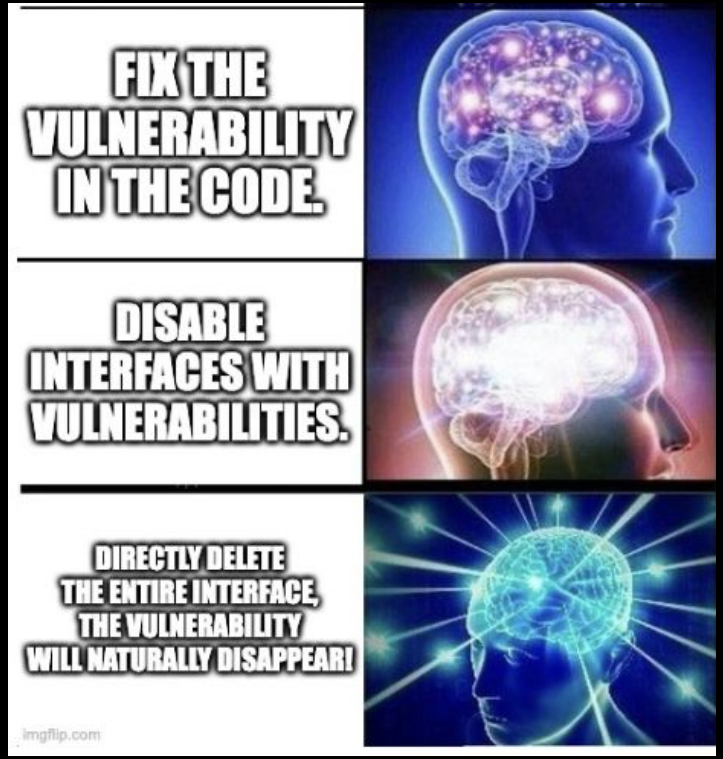
Chain: Patch the part of the chain.

mod_smbdav.so : APPLYAPP Vuln patch

In the updated version, several HTTP methods were removed, effectively fixing the vulnerability. These include:

```
HTTP_METHOD_GETNOTICE
HTTP_METHOD_GETLATESTVER
HTTP_METHOD_UPLOADTOFACEBOOK
HTTP_METHOD_UPLOADTOFLICKR
HTTP_METHOD_UPLOADTOPICASA
HTTP_METHOD_UPLOADTOTWITTER
HTTP_METHOD_APPLYAPP
HTTP_METHOD_NVRAMGET
```

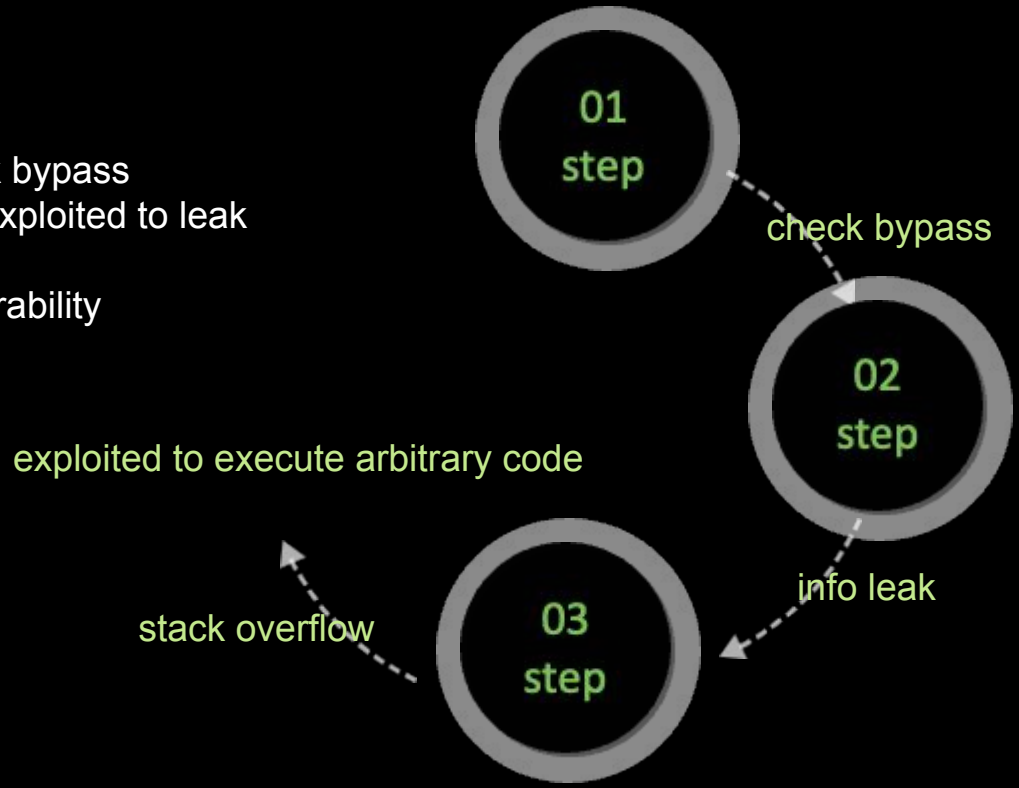
The removal of these methods eliminates the corresponding attack surfaces and addresses the previously exploitable vulnerabilities.



Vulnerability Chain Analysis: RWCTF 6th Router4 challenge

mod_webdav.so

- mod_webdav.so check bypass
- SQL injection can be exploited to leak memory addresses
- A stack overflow vulnerability



Chain: mod_webdav.so check bypass

In the lighttpd configuration file, the /smb($|/) URL path is defined to be handled by the WebDAV module:

```
else $HTTP["url"] =~ "^/smb($|/)" {
    server.document-root = "/"
    alias.url = ("/smb" => "/usr/lighttpd")
    smbdav.auth_ntlm = ("Microsoft-WebDAV", "xxBitKinex",
"WebDrive")
    webdav.activate = "enable"
    webdav.is-readonly = "disable"
}
```

This configuration enables the WebDAV module for the /smb path and its subdirectories, with the document root set to /usr/lighttpd. The WebDAV module is fully enabled, and the read-only flag is disabled, meaning both read and write operations are allowed.

Chain: mod_webdav.so check bypass

A check in mod_webdav_subrequest_handler returns 403 if the request targets /smb/ with a non-GET method.

```
// mod_webdav.c
URIHANDLER_FUNC(mod_webdav_subrequest_handler) {
        ...
    if(string_starts_with(con->url.path->ptr, "/smb/") &&
con->request.http_method!=HTTP_METHOD_GET){
        con->http_status = 403;
        return HANDLER_FINISHED;
    }
        ...
    switch (con->request.http_method) {
    case HTTP_METHOD_PROPFIND:
        ...
```

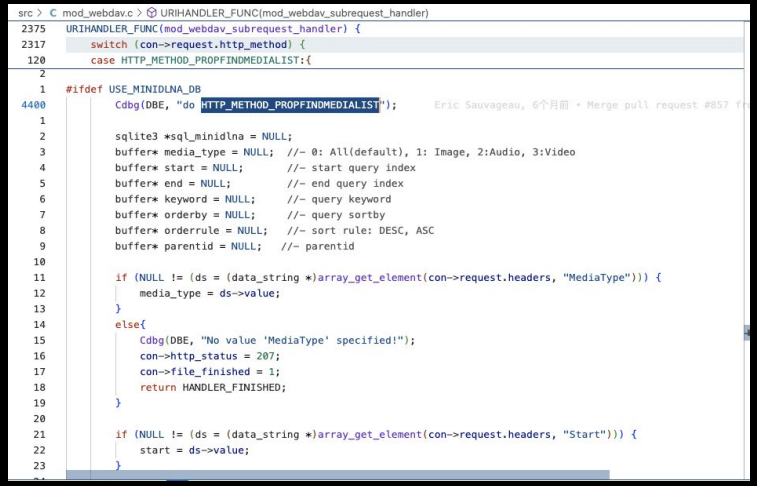
```
int string_starts_with( const char *a,
const char *b ){
    if(strncmp(a, b, strlen(b)) == 0)
return 1;
       return 0;
}
```

```
425  ds = (struct_ds *)array_get_element(a2->header, "Depth");
426  if ( ds )
427    v419 = strtol(*ds->value, 0, 10);
428  if ( !*(_WORD *)(a2->dword254 + 120) && (!*(_DWORD *)(a1 + 772) || a2->logged) )
429  {
430    a2->res_code = 403;
431    return 2;
432  }
433  if ( string_starts_with(*a2->uri, "/smb/") && a2->logged )
434  {
435    a2->res_code = 403;
436    return 2;
437  }
```

If the request path is /smb (without the trailing slash), the condition is not satisfied, allowing attackers to bypass the authorization logic.

mod_webdav.so mod_webdav_subrequest_handler

Within the mod_webdav.so module, the mod_webdav_subrequest_handler function processes several HTTP methods, such as HTTP_METHOD_PROPFINDMEDIALIST, to handle WebDAV-related requests and interactions.



Chain: mod_webdav.so SQL injection vulnerability

In the HTTP_METHOD_PROPFINDMEDIALIST method, the value of the keyword variable is extracted from the request header. It is directly concatenated into an SQL query using sprintf and executed.

```
case HTTP_METHOD_PROPFINDMEDIALIST:{
    ...
            if (NULL != (ds = (data_string *)array_get_element(con->request.headers, "Keyword"))) {
        keyword = ds->value;
    ...
            //- keyword : Avoid SQL injection!
    if (!buffer_is_empty(keyword) &&
        ((strchr(keyword->ptr, '\''))|| keyword->used > 200)) {
            ...
    }
    ...
    if(!buffer_is_empty(keyword)){

    if(strstr(keyword->ptr, "*")||strstr(keyword->ptr, "?")){
            char buff[200];
            replace_str(keyword->ptr, "*", "%", buff);
            replace_str(buff, "?", "_", buff);
            sprintf(sql_query, "%s and ( PATH LIKE '%s' or TITLE LIKE '%s' )", sql_query, buff,
buff);
        }
    else
            sprintf(sql_query, "%s and ( PATH LIKE '%s%s%s' or TITLE LIKE '%s%s%s' )", sql_query,
"%", keyword->ptr, "%", "%", keyword->ptr, "%");
    }
```

Chain: mod_webdav.so SQL injection vulnerability

Although there is a check in place:

```
2186  if ( !buffer_is_empty(keyword) && (strchr(*keyword, '\'') || (unsigned int)keyword[1] > 0xC8) )
2187  {
2188    dprintf_impl("mod_webdav.c", "mod_webdav_subrequest_handler", 4519, 1, "The paramter keyword is invalid!");
2189    a2->res_code = 207;
2190    a2->dword48 = 1;
2191    return 2;
```

it can be easily bypassed by using simple URL encoding techniques.



The Keyword is URL-decoded before being inserted into the SQL query, which enables attackers to bypass input validation using encoded payloads and still execute them after decoding.

Chain: mod_webdav.so SQL injection vulnerability

ASUS AiCloud's SQL operations are powered by SQLite3. The component relies on SQLite3 for handling database queries and storage,

```
admin@RT-AX86U-4D48:/tmp/home/root# ps w |grep lighttpd
 1661 admin    16508 S    /usr/sbin/lighttpd -f /tmp/lighttpd.conf -D
 1662 admin    12832 S    /usr/sbin/lighttpd-monitor
 2358 admin    13568 S    /usr/sbin/lighttpd-arpping -f br0
28934 admin     3424 S    grep lighttpd
admin@RT-AX86U-4D48:/tmp/home/root# cat /proc/1661/maps |grep sqlite
f7493000-f7592000 r-xp 00000000 00:0c 1839                               /usr/lib/libsqlite3.so.0.8.6
f7592000-f75a2000 ---p 000ff000 00:0c 1839                               /usr/lib/libsqlite3.so.0.8.6
f75a2000-f75a4000 r--p 000ff000 00:0c 1839                               /usr/lib/libsqlite3.so.0.8.6
f75a4000-f75a6000 rw-p 00101000 00:0c 1839                               /usr/lib/libsqlite3.so.0.8.6
admin@RT-AX86U-4D48:/tmp/home/root#
```

Chain: SQL Inject -> Base address leak

In SQLite3, there's a special function called fts3_tokenizer. This function is used to get the pointer to a custom tokenizer module.

```
if( sqlite3Fts2HashInsert(pHash, "simple", 7, (void *)pSimple)
 || sqlite3Fts2HashInsert(pHash, "porter", 7, (void *)pPorter)
```

For example, when using the `simple` tokenizer, its registered pointer points to the statically allocated `simpleTokenizerModule`.

```
static const sqlite3_tokenizer_module
simpleTokenizerModule = {
  0,
  simpleCreate,
  simpleDestroy,
  simpleOpen,
  simpleClose,
  simpleNext,
};
```

Chain: SQL Inject -> Base address leak

By leveraging the pointer to the simpleTokenizerModule, an attacker can calculate the base address of libsqlite3.so, effectively bypassing ASLR.



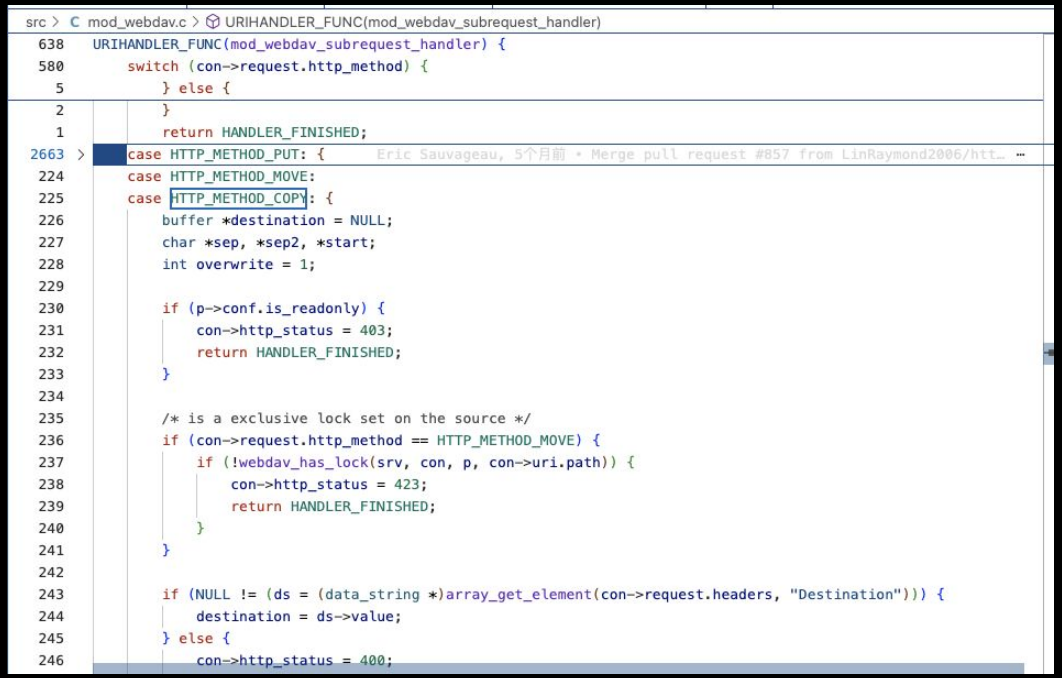

```
# root @ orbstack in /workhub [6:11:48] C:1
$ sqlite3
SQLite version 3.37.2 2022-01-06 13:25:41
Enter ".help" for usage hints.
Connected to a transient in-memory database.
Use ".open FILENAME" to reopen on a persistent database.
sqlite> select fts3_tokenizer('mytokenizer', x'0xdeadbeefdeadbeef');
Error: in prepare, unrecognized token: "x'0xdeadbeefdeadbeef'" (1)
sqlite> select hex(fts3_tokenizer('simple'));
40A4C7BC70550000
sqlite>
```

```
# root @ orbstack in /workhub [6:17:10]
$ cat /proc/180/maps |grep -E 5570
5570bcaef000-5570bcaff000 r--p 00000000 00:21 3740822
5570bcaff000-5570bcc30000 r-xp 00010000 00:21 3740822
5570bcc30000-5570bcc76000 r--p 00141000 00:21 3740822
5570bcc76000-5570bcc7b000 r--p 00186000 00:21 3740822
5570bcc7b000-5570bcc7f000 rw-p 0018b000 00:21 3740822
5570bcc7f000-5570bcc80000 rw-p 00000000 00:00 0
5570bd30a000-5570bd34c000 rw-p 00000000 00:00 0

# root @ orbstack in /workhub [6:17:22]
$
```

mod_webdav.so mod_webdav_subrequest_handler

Within the mod_webdav.so module, the mod_webdav_subrequest_handler function processes several HTTP methods, such as HTTP_METHOD_COPY, to handle WebDAV-related requests and interactions.



Chain: Stack Overflow

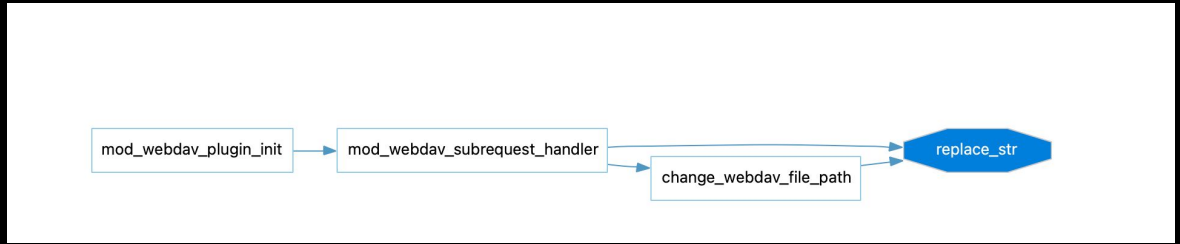
```
// mod_webdav.c
// mod_webdav_subrequest_hander
case HTTP_METHOD_MOVE:
case HTTP_METHOD_COPY: {
        ...
        if (NULL != (ds = (data_string *)array_get_element(con->request.headers,
"Destination"))) {
            destination = ds->value; }
            ...

        char buff[4096];
        memset(buff, 0, sizeof(buff));
        buffer_copy_string(p->uri.path, change_webdav_file_path(srv, con, p->uri.path->ptr,
buff)); // From Destination header
```

In the `HTTP_METHOD_COPY` method, the `change_webdav_file_path` function processes the `Destination`header, affecting file path handling for file operations.

Chain: HTTP_METHOD_MOVE(COPY) Stack Overflow

The change_webdav_file_path function calls replace_str, which handles the manipulation of the file path string, potentially introducing vulnerabilities.




```
const char *__fastcall sub_7E60(int a1, int a2, const char *a3, int buffer)
{
  // [COLLAPSED LOCAL DECLARATIONS. PRESS KEYPAD CTRL-"+" TO EXPAND]

  v10 = "alias.url";
  v11 = &dword_0;
  v12 = &byte_5;
  v13 = (int)&dword_0 + 2;
  v14 = &dword_0;
  v15 = &dword_0;
  v16 = &dword_0;
  v17 = &dword_0;
  for ( i = 1; *(_DWORD *)(*(_DWORD *)(a1 + 580) + 8) > i; ++i )
  {
    v20 = (int *)array_init();
    v11 = v20;
    if ( i )
      v4 = 2;
    else
      v4 = 1;
    if ( !config_insert_values_global(a1, *(_DWORD *)(*(_DWORD *)(**(_DWORD **)(a1 + 580) + 4 * i) + 32), &v10, v4) )
    {
      for ( j = 0; v20[2] > j; ++j )
      {
        v19 = *(_DWORD *)(*v20 + 4 * j);
        if ( !strncmp(a3, **(const char ***)(v19 + 4), *(_DWORD *)(*(_DWORD *)(v19 + 4) + 4) - 1) )
        {
          v18 = replace_str((int)a3, **(_DWORD **)(v19 + 4), **(_DWORD **)(v19 + 32), buffer);
          array_free(v20);
          return (const char *)v18;
        }
      }
      array_free(v20);
    }
  }
  return a3;
}
```

Chain: HTTP_METHOD_MOVE(COPY) Stack Overflow

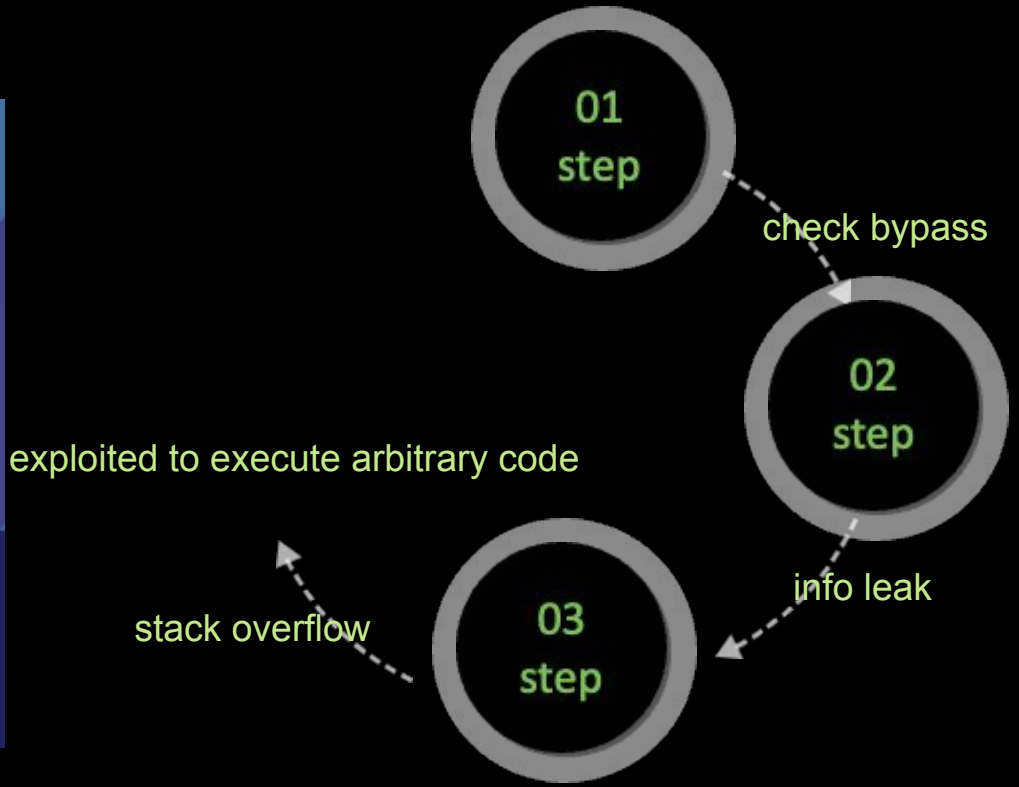
The replace_str function lacks length validation and uses sprintf to write into a buffer, leading to a potential stack buffer overflow vulnerability.

```
char *replace_str(char *st, char *orig, char *repl, char*
buff) {
    char *ch;
    if (!(ch = strstr(st, orig)))
        return st;
    strncpy(buff, st, ch-st);
    buff[ch-st] = 0;
    sprintf(buff+(ch-st), "%s%s", repl, ch+strlen(orig));

    return buff;
}
```

Vulnerability Chain Analysis: RWCTF 6th Router4 challenge





Chain: Patch the part of the chain.

After patch: Change /smb/ -> /smb

```
// mod_webdav.c
if(string_starts_with(con->url.path->ptr,
"/smb") && con->request.http_method !=
HTTP_METHOD_GET){
    con->http_status = 403;
    return HANDLER_FINISHED;
}
```

After patch: Add the parameter max_size to `replace_str`

```
char *__fastcall replace_str(const char *st, const char *orig, const char *repl, char *buff, unsigned int max_size)
{
  char *v9; // r0
  char *v10; // r5
  int v12; // r6
  char *v13; // r10
  size_t v14; // r6
  const char *v15; // r5

  v9 = strstr(st, orig);
  v10 = v9;
  if ( !v9 )
    return (char *)st;
  v12 = v9 - st;
  if ( v9 - st >= max_size )
    return (char *)st;
  v13 = &buff[v12];
  strncpy(buff, st, v9 - st);
  buff[v12] = 0;
  v14 = strlen(repl);
  v15 = &v10[strlen(orig)];
  if ( max_size <= v14 + strlen(v15) )
    return (char *)st;
  sprintf(v13, "%s%s", repl, v15);
  return buff;
}
```

```
  replace_str(v163, v162, **(_DWORD **)(a2 + 680), v523, 4096);
  buffer_copy_string(*(_DWORD *)(a3 + 16), v523);
}
```

Chain: mod_webdav.so SQL injection vulnerability

After patch:Decoding the cash URL before checking

```
Keyword_ptr = array_get_element(*(_DWORD *)(a2 + 304), "Keyword");
if ( Keyword_ptr )
{
  keyword = *(const char ***)(Keyword_ptr + 32);
  if ( !buffer_is_empty(keyword) )
    buffer_urldecode_path(keyword);
}
```

```
if ( buffer_is_empty(keyword) || !is_valid_string(*keyword) && (unsigned int)keyword[1] <= 0xC8 )
{
  sub_3664(a3);
  if ( !buffer_is_empty(*(_DWORD *)(a3 + 124)) && sqlite3_open(**(_DWORD **)(a3 + 124), &v515) )
    dprintf_impl(
      "mod_webdav.c",
      &unk_BFBD,
      4541,
      1,
      "Fail to open minidlna db %s",
      **(const char ***)(a3 + 124));
```

Vulnerability Summary (a.k.a. Why You Should Never Trust User Input)

- 🔐 Auth Check Bypass

AiCloud module says: "User-Agent is Microsoft-WebDAV. I thought you already authorized."

- 🧨 Command Injection

Some HTTP methods were like: "You gave me a string? Cool, let me system() that real quick."

- 🧠 ASLR Bypass via SQLite Magic

fts3_tokenizer('simple') leaks pointers → calculate base address → 💫 now you own the system.

- 🪜 Stack Overflow (Not the Website)

Functions like sprintf() + unchecked buffer = 💥 classic stack smashing combo.



Thanks



Q & A